

Universitatea Națională de Știință și Tehnologie
POLITEHNICA București
Facultatea de Automatică și Calculatoare
Departamentul de Calculatoare



PROIECT DE DIPLOMĂ

Un agent conversațional pentru Facultatea de Automatică și
Calculatoare

Hatu Denis-Alexandru

Conducători științifici:

S.I. ing. Șorici Alexandru

BUCUREȘTI

2026

CONTENTS

Listă de figuri	v
Listă de tabele	v
Abstract	v
1 Introducere	1
1.1 Context	1
1.2 Motivație	2
1.3 Enunțul problemei	2
1.4 Obiective	2
1.5 Cazuri de utilizare	3
1.5.1 Verificarea și transparența informațiilor	3
1.5.2 Suport în afara orelor de program	3
1.5.3 Gestionarea sesiunilor de conversație	4
1.5.4 Menținerea informațiilor recente	4
2 Stadiul actual	5
2.1 Agenți conversaționali (LLM)	5
2.2 Generare augmentată prin recuperare (RAG)	5
2.2.1 Indexarea vectorială	6
2.2.2 Recuperare densă	6
2.3 Reprezentări vectoriale pentru documente ipotetice (HyDE)	7
2.4 Colectarea datelor prin scanare web	7
2.4.1 Algoritmi de traversare și prioritzare	8
2.4.2 Politici de politețe și deduplicare	8
2.5 Agenți conversaționali în domeniul educațional	8

3	Arhitectura sistemului	9
3.1	Diagrama generală a sistemului	9
3.2	Tehnologii folosite	13
3.2.1	Limbaj de programare	13
3.2.2	Managementul pachetelor	13
3.2.3	Interfața utilizator	14
3.2.4	Model lingvistic	14
3.2.5	Modele de reprezentare vectorială	14
3.2.6	Baza de date vectorială	15
3.2.7	Scanare web	15
3.2.8	Extragerea conținutului HTML	15
3.2.9	Procesarea documentelor structurate	15
3.2.10	Infrastructura cloud	16
3.2.11	Deduplicarea URL-urilor	16
3.3	Infrastructura cloud Azure	16
3.4	Integrare cu UPB Login API	19
3.5	Fluxul de date	20
4	Implementare	22
4.1	Setările inițiale pentru contul de Azure Student	22
4.2	Colectarea datelor prin scanare web	22
4.2.1	Componentele Scrapy folosite	22
4.2.2	Setările Scrapy aplicate	23
4.2.3	Generarea unui agent utilizator dinamic	24
4.2.4	Folosirea unor certificate SSL vechi	24
4.2.5	Agentul de colectare web Scrapy	24
4.2.6	Extragerea documentelor	25
4.2.7	Extragerea textului	25
4.2.8	Extragerea tabelelor	25

4.2.9	Fluxul de prelucrare a datelor	25
4.3	Procesarea documentelor și a textului	26
4.3.1	Transformarea documentelor cu Docling și strategia de segmentare	26
4.3.2	Transformarea textului și strategia de segmentare	26
4.4	Codificare și indexare	27
4.5	Lanțul de recuperare (RAG)	27
4.5.1	Recuperarea densă	27
4.5.2	Re-ierarhizarea cu cross-encoder	28
4.5.3	Reformularea interogărilor	28
4.5.4	Atomicizarea interogărilor	28
4.5.5	Generarea unor documente sintetice	29
4.6	Detectarea schimbării documentelor	29
4.7	Generarea răspunsurilor	29
4.8	Gestionarea contextului	30
4.9	Gestionarea conversațiilor	30
4.10	Sistemul de autentificare	31
4.11	Interfața utilizator	31
5	Provocări întâmpinate	32
5.1	Provocări legate de fluxul de recuperare al datelor	32
5.2	Provocări în transformarea și normalizarea textului și a tabelelor	32
5.3	Provocări în parcurgerea site-urilor web	33
5.4	Optimizarea fluxului de recuperare	33
6	Metodologie de evaluare	34
6.1	Rezultatele evaluării	34
6.2	Limitări	34
7	Concluzii și muncă viitoare	35
7.1	Concluzii	35

7.2	Muncă viitoare	36
7.2.1	Dezvoltarea unei interfețe utilizator moderne	36
7.2.2	Integrare cu toate site-urile web ale facultății	36
7.2.3	Infrastructură proprie față de API	36
7.2.4	Integrare cu platforma Microsoft Teams	36
7.2.5	Integrare cu platforma de cursuri Moodle	36
7.2.6	Integrare cu platforma studenti.pub.ro	37
7.2.7	Detectare în timp real a schimbării conținutului documentelor	37
7.2.8	Folosirea imaginilor în momentul generării răspunsului	37
	Referințe	38

LISTĂ DE FIGURI

1	Fluxul zilnic de colectare a datelor: de la inițializarea agentului de scanare până la stocarea fragmentelor în baza de date vectorială	10
2	Fluxul de procesare a interogărilor: reformulare și normalizare, generare document ipotetic (HyDE), recuperare vectorială, re-ierarhizare cu CrossEncoder și generare răspuns prin Claude API	12
3	Fluxul de autentificare prin UPB Login API (OAuth 2.0 / Keycloak)	20

LISTĂ DE TABELE

1	Comparație tehnologii alese față de alternative	17
2	Serviciile Azure utilizate și integrarea cu proiectul	19

REZUMAT

Lucrarea de față prezintă proiectarea și implementarea unui agent conversațional pentru Facultatea de Automatică și Calculatoare din cadrul Universității Politehnica București, destinat răspunderii automate a întrebărilor administrative ale studenților. Sistemul se bazează pe generare augmentată prin recuperare (RAG), combinând un corpus de documente oficiale colectat zilnic prin scanare web cu modele lingvistice de mari dimensiuni pentru generarea răspunsurilor.

Corpusul este construit printr-un agent de colectare Scrapy care parcurge sursele oficiale ale facultății și descarcă documente în formatele PDF, DOCX, XLSX și XLS. Conținutul extras este segmentat prin HybridChunker din biblioteca Docling, iar fragmentele sunt indexate în baza de date vectorială ChromaDB cu modelul de codificare `intfloat/multilingual-e5-large`.

Fluxul de interogare aplică patru etape de preprocesare: normalizare Unicode, rezolvarea referințelor pronominale din întrebările de urmărire, rescriere academică și descompunerea înterogărilor compuse în sub-interogări independente. Pentru fiecare sub-interogare, sistemul generează un document ipotetic în stilul surselor facultății prin metoda HyDE (*Hypothetical Document Embedding*), al cărui vector este utilizat pentru căutarea vectorială densă. Rezultatele recuperate sunt reclasificate prin cross-encoderul BAAI/bge-reranker-v2-m3, iar primele zece fragmente sunt transmise modelului `claude-sonnet-4-6` pentru generarea răspunsului final.

Interfața web, construită cu Streamlit, gestionează autentificarea prin UPB Login API, istoricul conversațiilor cu titluri și rezumate generate automat și afișarea surselor utilizate pentru fiecare răspuns. Infrastructura este găzduită pe Microsoft Azure prin Azure Container Apps.

Cuvinte cheie: Modele Lingvistice, Procesarea Limbajului Natural, Generare Augmentată prin Recuperare, Agent Conversațional, Scanare web

1 INTRODUCERE

1.1 Context

Multe secretariate universitare se confruntă cu un deficit de personal disponibil să răspundă la întrebările studenților, sarcina fiind pusă frecvent pe umerii profesorilor. Volumul solicitărilor administrative depășește adesea capacitatea de răspuns, ceea ce generează întârzieri și incertitudine pentru studenții care au nevoie de informații urgente.

Facultatea de Automatică și Calculatoare gestionează un volum mare și recurent de întrebări despre înmatriculare, orar și regulamente universitare. Pentru aceste tipuri de întrebări se poate prototipa un agent conversațional specializat pe aceste tipuri de solicitări fără a necesita intervenția directă a personalului.

Un factor care amplifică dificultatea este distribuirea informațiilor pe mai multe surse care nu sunt centralizate: site-ul facultății, platforma de cursuri Moodle, portalul studenti.pub.ro și documentele oficiale publicate în formate variate (PDF, DOCX). Un student care dorește să afle, de exemplu, numărul de credite al unei materii sau termenul de depunere a unei cereri trebuie să știe în prealabil platforma în care se află informația respectivă. Lipsa informațiilor unificate crește efortul de căutare și generează solicitări repetate către secretariat pentru informații care sunt, în principiu, publice.

Stadiul actual al tehnologiei permite automatizarea acestui proces cu ajutorul inteligenței artificiale, cu potențialul de a reduce substanțial volumul de muncă de la secretariat. Modelele lingvistice de mari dimensiuni, combinate cu tehnici de recuperare semantică, oferă mijloacele necesare pentru a construi sisteme capabile să răspundă întrebărilor formulate în limbaj natural, ancorate în documente instituționale specifice.

Implementarea asistenților virtuali în mediul academic a fost documentată atât la nivel european, cât și global. Un exemplu este Jill Watson, dezvoltat în cadrul Georgia Institute of Technology, inițial pe platforma IBM Watson și ulterior migrat pe GPT-4. Sistemul a fost utilizat în cadrul cursului de Inteligență Artificială din programul online OMSCS și a fost evaluat pe un set de 150 de întrebări, atingând o rată de răspunsuri corecte de 76.7%, față de 31.3% pentru OpenAI Assistants (1). La nivel european, ETH Zurich a introdus asistentul virtual Ethel, bazat pe recuperare augmentată și GPT-4, utilizat în cadrul a șase cursuri cu peste 1.500 de studenți; studenții au evaluat răspunsurile ca fiind utile în aproximativ trei sferturi din cazuri (2). Pe baza celor două lucrări se are în vedere constituirea unui cadru de referință pentru lucrarea de față.

1.2 Motivație

Reducerea sarcinii administrative a personalului de secretariat este o necesitate reală în contextul universitar actual, în care volumul de solicitări depășește frecvent capacitatea de răspuns. Un sistem care preia întrebările frecvente și repetitive eliberează personalul pentru activități care necesită atenție sporită, cu efect direct asupra eficienței secretariatului.

Studentii au nevoie de acces rapid și precis la informații, fără a fi nevoiți să parcurgă documente voluminoase sau să aștepte programul de lucru de la secretariat. În cadrul Facultății de Automatică și Calculatoare se poate demonstra că un agent conversațional disponibil permanent, bazat pe procesarea limbajului natural este fezabil acestei nevoi.

1.3 Enunțul problemei

Studentii nu găsesc adesea răspunsuri clare la întrebări administrative simple în afara orelor de program, ceea ce duce la întârzieri și incertitudine. Lipsa unui punct unic de acces la informațiile oficiale îi obligă să caute pe mai multe platforme sau să aștepte deschiderea secretariatului, chiar și pentru solicitări banale.

Asistenții virtuali de uz general, deși capabili să răspundă la întrebări generice, nu sunt ancorați în documentele, regulamentele și terminologia specifice facultății. Fără acces la sursele instituționale actualizate, aceste sisteme produc răspunsuri care par corecte, dar care pot fi cu greu verificate de student și nu reflectă reglementările în vigoare.

Prin urmare, se urmărește construirea unui instrument de inteligență artificială bazat pe recuperare de informații, actualizat periodic cu cele mai recente date instituționale colectate direct de pe sursele oficiale ale facultății. Sistemul va oferi răspunsuri verificabile, însoțite de referințele documentelor din care au fost extrase, eliminând ambiguitatea specifică asistenților de uz general.

O problemă suplimentară este reprezentată de actualizarea frecventă a informațiilor: orarele, termenele și regulamentele se modifică frecvent, iar studenții nu au o unealtă care să îi notifice cu privire la modificările făcute. Colectarea automată și periodică a surselor oficiale asigură că sistemul reflectă întotdeauna starea curentă a documentelor instituționale.

1.4 Obiective

Construirea unui sistem bazat pe recuperare ancorat în documentele oficiale ale facultății, cu răspunsuri verificabile și limitate la surse autorizate. Scopul sistemului este a suplini prezența de personal în afara orelor de program.

Implementarea diverselor tehnici de reformulare și augmentare semantică pentru a reduce

diferența dintre limbajul colocvial al studenților și formalitatea documentelor oficiale. Studenții formulează întrebările în mod liber, cu greșeli de scriere sau fără diacritice, în timp ce documentele instituționale folosesc un limbaj tehnic și standardizat; această discrepanță trebuie rezolvată înainte de etapa de căutare.

Măsurarea acurateței răspunsurilor pe un set de întrebări administrative cu răspuns unic determinat, cu scopul de a verifica dacă sistemul este mai specific și mai fiabil decât un asistent de uz general care nu are acces la sursele instituționale.

Dezvoltarea unei interfețe conversaționale dedicată studenților, adaptată nevoilor lor specifice și intuitiv de utilizat, fără a fi necesar de cunoștințe tehnice avansate în prealabil.

1.5 Cazuri de utilizare

1.5.1 Verificarea și transparența informațiilor

Interfața afișează, pentru fiecare răspuns, un panou cu referințele documentelor oficiale din care au fost extrase fragmentele relevante pentru contextul întrebării. Studentul poate accesa sursa primară a informației direct din interfață, fără a fi nevoit să caute manual documentul original.

În situațiile în care niciun fragment recuperat din baza de date vectorială nu depășește pragul setat de încredere, sistemul declară explicit că nu poate răspunde pe baza surselor disponibile, evitând astfel răspunsuri inventate despre proceduri sau termene. Transparența este esențială într-un context academic, unde o informație incorectă despre un termen sau o cerință poate genera incertitudine.

Studentul poate astfel distinge clar între un răspuns cert, susținut de surse indexate, și o situație în care întrebarea depășește corpusul disponibil, cu îndrumare explicită spre secretariat pentru clarificări. Sistemul devine astfel un prim nivel de filtrare: numărul de solicitări adresate personalului scade, iar secretariatul primește numai cazurile cu adevărat complexe.

1.5.2 Suport în afara orelor de program

Sistemul este accesibil oricând printr-o interfață web, oferind răspunsuri imediate la întrebările administrative frecvente privind termene, orar și regulamente.

Conversațiile anterioare sunt salvate automat și pot fi reluate la o autentificare ulterioară; fiecărei conversații i se atribuie un titlu și un rezumat generate automat prin Claude API, permițând studentului să identifice rapid o discuție anterioară fără a citi întregul istoric.

Din perspectiva securității, sesiunea expiră automat după o perioadă de inactivitate, fără a solicita acțiuni suplimentare din partea studentului, asigurând protecția datelor de autentifi-

care.

1.5.3 Gestionarea sesiunilor de conversație

Mecanismul de autentificare prin UPB Login API garantează că accesul este restricționat exclusiv membrilor comunității universitare.

1.5.4 Menținerea informațiilor recente

Prin utilizarea unei unelte specifice pentru Facultatea de Automatică și Calculatoare se poate păstra și folosi cea mai recentă informație.

2 STADIUL ACTUAL

2.1 Agenți conversaționali (LLM)

Agenții conversaționali sunt sisteme software care integrează modele de limbaj de mari dimensiuni pre-antrenate, configurate prin instrucțiuni de sistem pentru a răspunde interogărilor utilizatorilor într-un domeniu definit. La primirea unei interogări, modelul procesează atât instrucțiunile de sistem, cât și mesajul utilizatorului, după care un set de unelte interne poate fi apelat, dacă este necesar, în scopul producerii unui răspuns adecvat. Primele sisteme de dialog automat sunt prezente încă din anii 1960. ELIZA (3), un program bazat pe potrivire de șabloane și substituție de text, simula conversația prin recunoașterea unor structuri sintactice predefinite; răspunsurile erau generate exclusiv prin transformări simple, fără memorie contextuală..

Introducerea modelelor neuronale a schimbat abordarea procesării limbajului natural. Bengio et al. (4) au propus primul model probabilistic neuronal de limbaj și au arătat că reprezentările distribuite ale cuvintelor pot captura cuvinte comune statistice limbii. Rețelele recurente LSTM (5) au permis modelarea dependențelor secvențiale, însă sufereau de problema gradientului care dispare pe secvențe lungi, ceea ce limita captarea contextului semantic de lungă durată.

Arhitectura Transformer (6) a înlocuit recurența cu mecanismul de atenție (self-attention), ceea ce a permis procesarea paralelă și captarea dependențelor globale. Pe baza acestui fapt au fost dezvoltate modele pre-antrenate pe corpusuri masive: BERT (7), orientat spre înțelegerea bidirecțională a limbii, și GPT (8), bazat pe generare autoregresivă.

Modelele de limbaj de mari dimensiuni (LLM) sunt extensia acestei paradigme. Brown et al. (9) au demonstrat cu GPT-3 că modelele la scară suficient de mare pot realiza învățare cu puține exemple fără reantrenare. Wei et al. (10) au descoperit comportamente calitative noi apărute odată cu creșterea dimensiunii modelului. Ouyang et al. (11) au introdus alinierea prin sugestii umane (RLHF), esențială pentru adaptarea LLM-urilor la dialog instrucțional.

2.2 Generare augmentată prin recuperare (RAG)

RAG este o metodă de îmbunătățire a modelelor generative prin recuperare la momentul inferenței a informațiilor relevante dintr-o bază de date vectorială înainte ca răspunsul să fie generat.

Un sistem bazat pe recuperare este alcătuit din trei componente principale:

Componenta de recuperare indexează documentele sub formă de reprezentări numerice ale conținutului semantic, și identifică fragmentele cu cel mai ridicat grad de relevanță față de interogarea utilizatorului prin metoda de similitudine aleasă.

Componenta de augmentare construiește instrucțiunea finală prin combinarea cererii utilizatorului cu fragmentele recuperate.

Componenta de generare produce răspunsul pe baza instrucțiunii augmentate, utilizând modelul lingvistic configurat.

2.2.1 Indexarea vectorială

Indexarea vectorială este procesul prin care documentele de tip text sunt transformate în reprezentări numerice, denumite vectori, printr-un model de reprezentare semantică. Fiecare document sau fragment de text este proiectat într-un spațiu vectorial, iar dimensiunea fiecărui vector este fixată de arhitectura modelului de codificare ales.

Proprietatea spațiului vectorial este că documentele cu conținut semantic similar sunt poziționate aproape unele de altele, indiferent de forma lor lexicală exactă. Vectorii rezultați sunt stocați într-o bază de date vectorială care permite căutarea prin diverse metode de similitudine.

Malkov și Yashunin propun algoritmul Hierarchical Navigable Small World (HNSW) (12), o formă de index pe mai multe niveluri bazată pe grafuri de proximitate. La fiecare nivel, nodul curent este conectat la cei mai apropiați vecini, iar căutarea pornește de la nivelul cel mai înalt și coboară treptat înspre niveluri din ce în ce mai dense, reducând complexitatea de căutare de la liniară la logaritmică. HNSW oferă un compromis favorabil între viteza de interogare, consumul de memorie și precizia rezultatelor, ceea ce l-a consacrat ca algoritm implicit în bazele de date vectoriale moderne.

2.2.2 Recuperare densă

Recuperarea densă este o metodă de identificare a documentelor relevante dintr-o colecție, prin compararea reprezentărilor vectoriale ale interogării cu cele ale documentelor indexate. Interogarea utilizatorului este codificată cu același model de reprezentare semantică folosit la indexare, obținând un vector de aceeași dimensiune cu cea a documentelor stocate.

Spre deosebire de metodele tradiționale bazate pe frecvența termenilor, cum este BM25, recuperarea densă calculează similaritatea semantică directă între interogare și fiecare document prin operații în spațiul vectorial, cel mai frecvent prin similitudine cosinus. Această proprietate permite identificarea documentelor relevante chiar și atunci când interogarea și documentul nu au niciun cuvânt comun.

Karpukhin et al. introduc Dense Passage Retrieval (DPR) (13), demonstrând că vectorii denși antrenați prin codificatoare duale specializate depășesc metodele bazate pe frecvența termenilor legat de răspuns la întrebări pe domeniu deschis. DPR utilizează două codificatoare BERT independente, unul pentru interogări și unul pentru pasaje, antrenate prin maximizarea similarității cosinus între perechi relevante și minimizarea similarității față de pasaje negative. Rezultatele demonstrează că recuperarea densă poate fi superioară metodelor lexicale chiar și în absența potrivirii exacte de termeni.

2.3 Reprezentări vectoriale pentru documente ipotetice (HyDE)

Reprezentările vectoriale pentru documente ipotetice (HyDE) sunt o tehnică de transformare a interogărilor introdusă de (14), concepută pentru a îmbunătăți regăsirea densă în scenarii fără date de antrenare etichetate, fără a necesita etichete de relevanță. Mecanismul central funcționează în două etape: mai întâi, un model de limbaj cu urmărire de instrucțiuni (de exemplu, InstructGPT) generează un document ipotetic ca răspuns la interogarea utilizatorului; deși documentul captează tipare de relevanță semantică, acesta poate conține detalii factuale incorecte. Ulterior, un codificator nesupervizat codifică documentul generat într-un vector de reprezentare, utilizat pentru regăsirea documentelor reale din corpus prin similaritate cosinus. Această tehnică acționează ca un filtru, eliminând erorile factuale și păstrând relevanța semantică. Experimental, HyDE depășește sistemele de recuperare densă nesupervizate de referință și obține performanțe comparabile cu sistemele antrenate cu etichete de relevanță pe sarcini diverse, inclusiv regăsire web, răspuns la întrebări și verificare de fapte (14).

În contextul chatbot-urilor bazate pe RAG, (15) propune un cadru avansat care integrează HyDE cu un model Gemma 7B, obținând îmbunătățiri măsurabile ale preciziei de regăsire, relevanței contextuale și fluenței răspunsurilor. Generarea unui raport ipotetic intermediar reduce decalajul semantic dintre interogarea brută a utilizatorului și documentele din corpus, permițând sistemului să recupereze documente specifice domeniului aliniate cu intenția reală a utilizatorului.

2.4 Colectarea datelor prin scanare web

Colectarea datelor prin scanare web este procesul automatizat de extragere a conținutului de pe paginile web, realizat prin agenți de colectare. Un agent de colectare pornește de la un set de adrese URL inițiale, urmărește recursiv legăturile descoperite în paginile vizitate și descarcă conținutul fiecărei pagini accesate, cu respectarea regulilor de acces definite de prin fișierele `robots.txt` prezente în sursele respective.

Conținutul extras trece prin etape de deduplicare și normalizare, după care poate fi utilizat în diverse aplicații: indexare vectorială, antrenarea modelelor bazate pe limbaj natural. Spre deosebire de accesul manual, scanarea web permite colectarea unui volum mare de date sub

diverse forme, inclusiv text, tabele și documente atașate, într-un interval scurt de timp.

2.4.1 Algoritmi de traversare și prioritizare

Primele sisteme de scanare web foloseau strategii de traversare în lățime (BFS, *breadth-first search*), pornind de la premisa că paginile descoperite devreme tind să fie mai relevante (16). Strategia BFS s-a dovedit eficientă pentru acoperire generală, însă nu optimizează calitatea paginilor colectate în funcție de un criteriu tematic. Ca alternativă, traversarea în adâncime (DFS) este rar utilizată în scanarea web la scară largă, deoarece riscă blocarea agentului în porțiuni nereprezentative ale grafului web.

2.4.2 Politici de politețe și deduplicare

Respectarea resurselor serverelor vizitate este reglementată prin fișierele `robots.txt` și prin directivele `Crawl-delay`, care limitează frecvența de acces. Dincolo de aceste convenții, crawlerele responsabile implementează politici proprii de curtoazie, cum ar fi un interval minim configurabil între cereri consecutive către același server.

Deduplicarea conținutului colectat se realizează la două niveluri: deduplicare exactă, prin compararea hash-urilor criptografice (MD5, SHA) ale conținutului descărcat, și deduplicare aproximativă, pentru detectarea conținutului cvasi-identic. MinHash combinat cu Locality-Sensitive Hashing (LSH) (17) permite estimarea similarității Jaccard între seturi de n -grame la scară masivă cu complexitate sublinară față de dimensiunea corpusului.

2.5 Agenți conversaționali în domeniul educațional

- to do

O căutare în literatura de specialitate efectuată în cadrul acestei lucrări nu a identificat niciun sistem disponibil public care să vizeze specific conținutul administrativ din învățământul superior românesc. Sistemele documentate în literatură se concentrează pe suport academic pentru cursuri individuale sau pe răspunsuri la întrebări de conținut disciplinar, nu pe proceduri administrative instituționale.

3 ARHITECTURA SISTEMULUI

3.1 Diagrama generală a sistemului

Agentul conversațional este structurat pe componente distincte, având un flux de desfășurare secvențial. Sistemul funcționează pe baza a două fluxuri de date complementare: fluxul de date zilnic, responsabil de actualizarea bazei de date vectoriale, și fluxul declanșat la momentul fiecărei interogări primite din interfață din partea utilizatorului.

Fluxul de date zilnic este ilustrat în Figura 1. SpiderRunner inițializează un proces Scrapy pornind de la o listă de URL-uri reprezentând sursele oficiale ale facultății și golește baza de date Redis pentru a reseta starea de deduplicare înainte de începerea parcurgerii. Această resetare garantează că fiecare rulare zilnică procesează toate sursele de la zero, indiferent de starea anterioară.

PageSpider parcurge paginile site-ului facultății, extrage conținutul HTML și descarcă documentele atașate (PDF, DOCX, XLS, XLSX). ValidationPipeline normalizează URL-urile, iar DeduplicationPipeline elimină intrările duplicate după URL și hash de conținut, prevenind indexarea redundantă a aceluiași conținut.

ContentChunkingPipeline transformă conținutul în format Markdown și îl împarte în fragmente de text și tabel. Asupra conținutului este aplicată o tehnică de semi-deduplicare prin care fragmentele similare sunt identificate și filtrate, astfel încât ChromaDB stochează exclusiv conținut unic. DocumentFilesPipeline se ocupă de descărcarea documentelor și aplică același proces de fragmentare și stocare pentru fișierele identificate în cursul scanării.

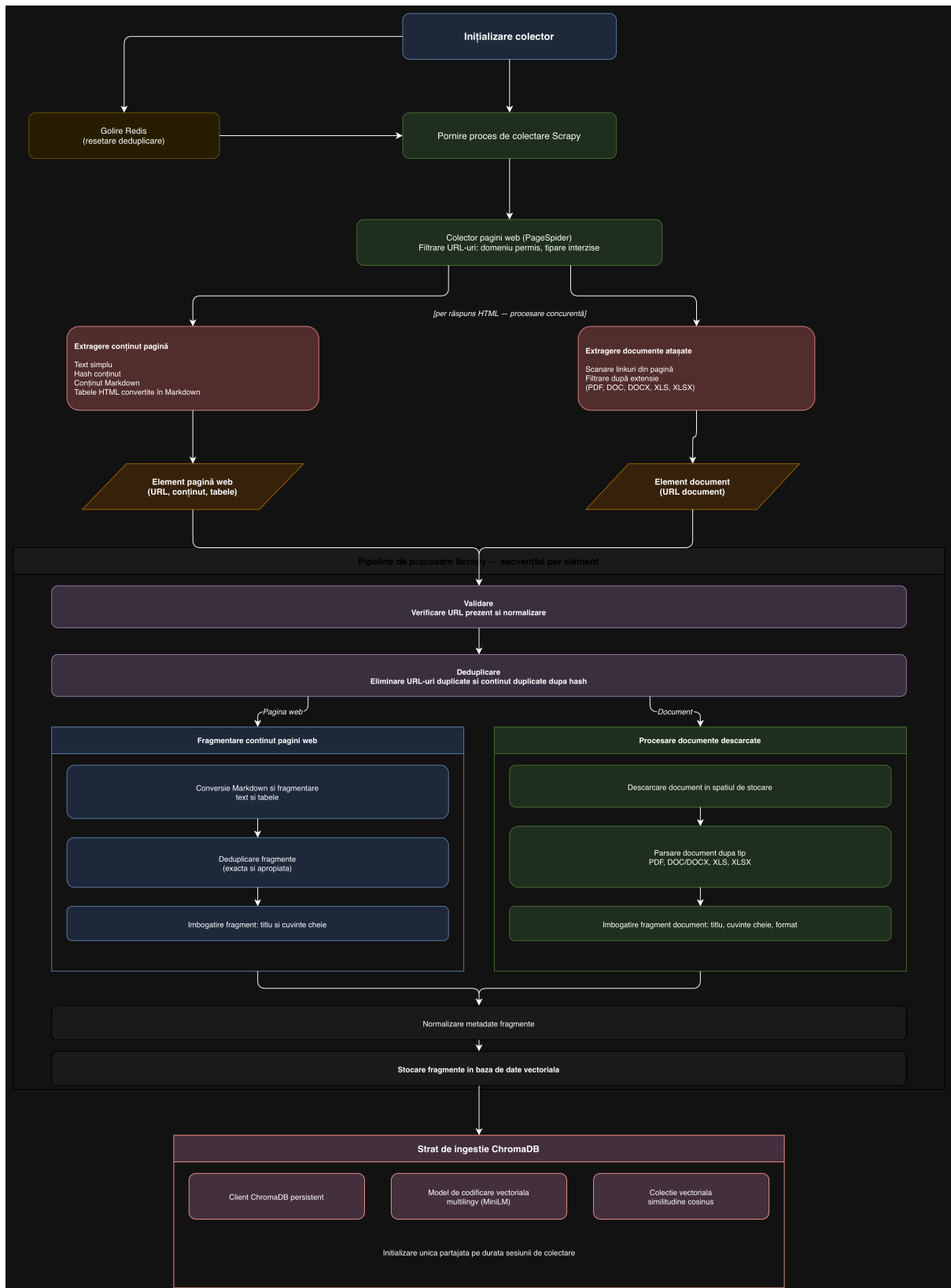


Figure 1: Fluxul zilnic de colectare a datelor: de la inițializarea agentului de scanare până la stocarea fragmentelor în baza de date vectorială

Fluxul declanșat la momentul interogării este ilustrat în Figura 2. Interogarea utilizatorului este procesată de un modul de preprocesare care aplică normalizarea Unicode, rescrierea textului și descompunerea cererilor compuse în sub-interogări independente, fiecare cu semnificație semantică de sine stătătoare.

Fiecare sub-interogare este transmisă modulului de recuperare, care interoghează baza de date vectorială ChromaDB pe baza unui document ipotetic generat prin HyDE (*Hypothetical Document Embedding*). Fragmentele recuperate sunt filtrate după un prag de relevanță și reclasificate prin CrossEncoder înainte de a fi selectate pentru generare.

Fragmentele finale, împreună cu interogarea originală, sunt transmise lui Claude API prin intermediul unei instrucțiuni de sistem configurate specific pentru domeniul universitar. Răspunsul este redat în interfață însoțit de referințele surselor utilizate.

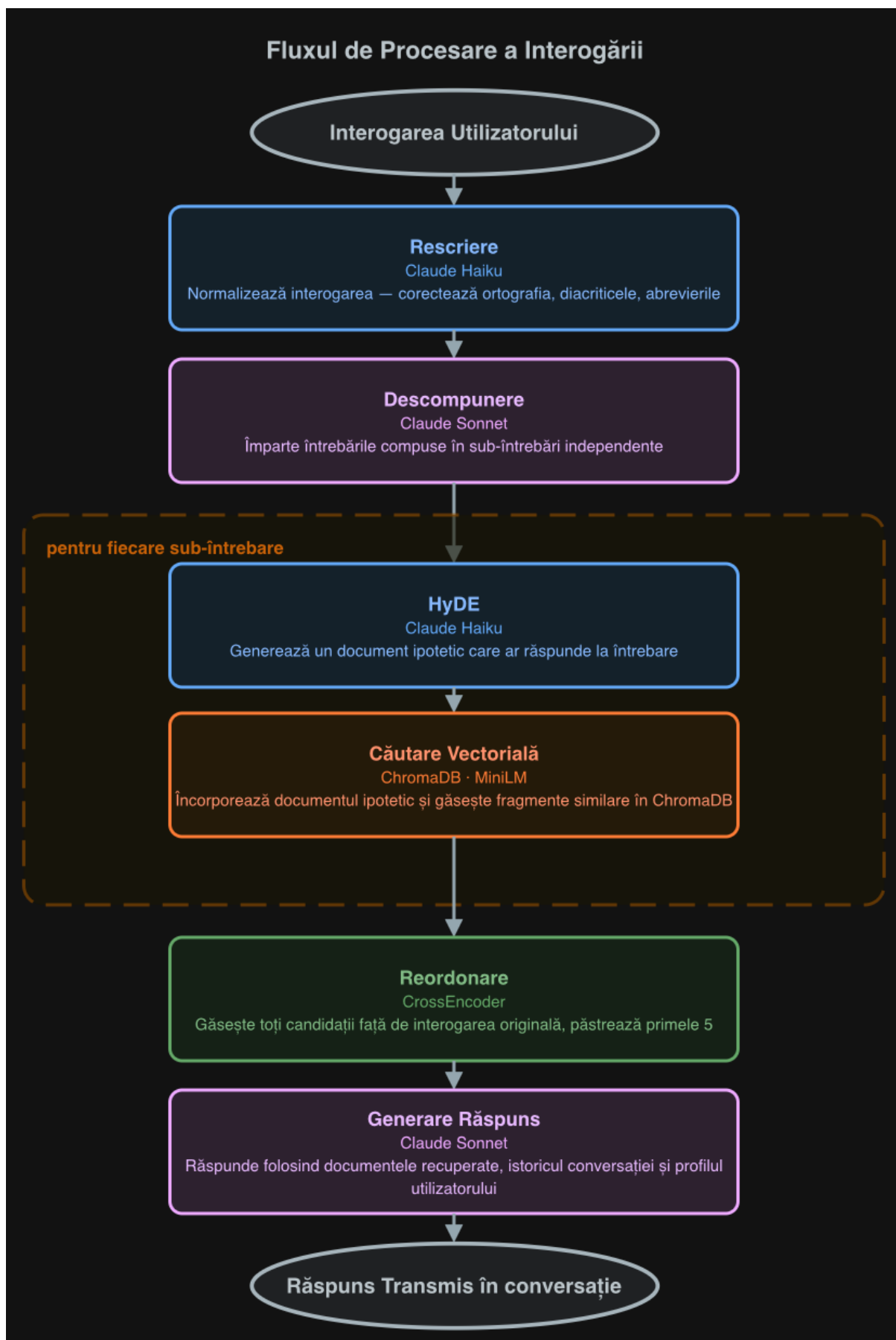


Figure 2: Fluxul de procesare a interogărilor: reformulare și normalizare, generare document ipotetic (HyDE), recuperare vectorială, re-¹²ierarhizare cu CrossEncoder și generare răspuns prin Claude API

3.2 Tehnologii folosite

Întreaga dezvoltare software a fost realizată pe un MacBook M4 Air cu procesor ARM cu 10 nuclee.

3.2.1 Limbaj de programare

Ca limbaj de programare s-a ales Python 3.14. Proiectul vizează mai multe arii distincte: procesarea limbajului natural, tehnici de învățare automată și scanare web. TypeScript ar fi fost o alegere bună, dar este orientat spre dezvoltarea aplicațiilor web și de server și nu acoperă complet niciuna dintre ariile enumerate.

Disponibilitatea bibliotecilor specializate a reprezentat un factor important. Scrapy și scrapy-redis nu au echivalente în TypeScript pentru scanarea web cu deduplicare prin Redis. Bibliotecile de recuperare augmentată, precum LangChain și ChromaDB, sunt native Python și nu au suport oficial complet în TypeScript. Modelele sentence-transformers sunt încărcate direct în memorie disponibile în biblioteca sentence-transformers, indisponibilă în TypeScript; utilizarea lor ar fi necesitat un serviciu intermediar dedicat, adăugând complexitate arhitecturală fără beneficiu funcțional.

Integrarea cu serviciile Azure utilizate în proiect este acoperită complet de uneltele Python oficiale, în timp ce suportul TypeScript pentru servicii precum Azure Table Storage sau Azure Communication Services este mai limitat.

3.2.2 Managementul pachetelor

Ca manager de pachete s-a utilizat uv. Spre deosebire de pip, uv nu necesită o instalare Python preexistentă și înlocuiește cu o singură unealtă întregul lanț pip + pip-tools + virtualenv + pyenv. Viteza de instalare este semnificativ mai mare față de un mediu virtual clasic cu pip, ceea ce este relevant atunci când mediul trebuie reprodus într-un alt mediu.

uv generează un fișier uv.lock care fixează toate dependențele tranzitive, nu doar cele directe. Dacă o bibliotecă precum langchain depinde la rândul ei de alte pachete, uv garantează că versiunile acestora sunt identice la fiecare instalare. Un fișier requirements.txt clasic nu oferă aceeași garanție: două instalări succesive pot produce versiuni diferite ale dependențelor tranzitive, ceea ce afectează reproductibilitatea mediului de execuție.

3.2.3 Interfața utilizator

Pentru interacțiunea cu utilizatorul s-a utilizat `Streamlit`, o bibliotecă Python pentru construirea interfețelor web. Componentele native `st.chat_message` și `st.chat_input` oferă o interfață de tip conversațional ușor de utilizat, fără a necesita implementarea manuală a bulelor de mesaje și a câmpului de introducere a textului. Redarea răspunsurilor dinamic este posibilă prin `st.write_stream`, permițând afișarea cuvânt cu cuvânt a răspunsului generat de Claude API, fără a bloca interfața până la finalizarea completă a răspunsului.

Gestionarea stării sesiunii se realizează prin `st.session_state`, un dicționar persistent pe durata unei sesiuni de browser, care stochează istoricul conversației, identitatea utilizatorului autentificat și conversația activă. Fiecare interacțiune a utilizatorului declanșează o reluare completă a scriptului, iar `session_state` asigură că datele anterioare nu se pierd între rerulări.

O implementare bazată pe `React` sau `Next.js` ar fi adăugat un strat de complexitate care nu aduce valoare în contextul unui prototip academic, al cărui scop este acuratețea răspunsurilor și transparența surselor. Integrarea aplicației `Streamlit` se realizează cu un singur fișier `Dockerfile`, fără configurare suplimentară de server web (`nginx`, `gunicorn`), ceea ce simplifică procesul de urcare a aplicației pe `Azure Container Apps`.

3.2.4 Model lingvistic

Pentru generarea răspunsurilor s-au integrat modelele `claude-sonnet-4-6` și `claude-haiku-4-5-20251001` prin Claude API. Alegerea față de alternative precum `OpenAI` sau `Google Gemini` a fost motivată de fereastra de context extinsă, necesară procesării documentelor care nu au suport în `Docling`, și de performanța demonstrată în urmărirea instrucțiunilor complexe și specifice domeniului academic. Cele două modele acoperă complementar cerințele sistemului: `claude-haiku-4-5-20251001` este utilizat pentru operațiile repetate (reformularea și decompunerea interogărilor), iar `claude-sonnet-4-6` pentru generarea răspunsurilor finale și procesarea documentelor.

3.2.5 Modele de reprezentare vectorială

Pentru generarea reprezentărilor vectoriale s-au utilizat modelul `intfloat/multilingual-e5-large` pentru codificarea fragmentelor și interogărilor, și `BAAI/bge-reranker-v2-m3` ca model de re-ierarhizare. Limba română este inclusă în datele de antrenare ale ambelor modele. Arhitectura multilingvă permite inferența fără GPU dedicat, ceea ce se potrivește constrângerilor de resurse ale infrastructurii `Azure` utilizate.

3.2.6 Baza de date vectorială

Ca bază de date vectorială s-a ales ChromaDB, cu index HNSW și distanță cosinus, găzduită pe aceeași instanță Azure și integrată prin langchain-chroma. Pinecone este un serviciu administrat extern, ceea ce ar fi presupus stocarea datelor în afara infrastructurii Azure a proiectului, cu un cost de \$70 - \$300 pe lună pentru planul standard. Elasticsearch este în primul rând un motor de căutare bazat pe indexul BM25; suportul pentru căutare vectorială a fost adăugat ulterior ca funcționalitate secundară, astfel că un instrument orientat spre căutarea lexicală nu se potrivește unui sistem care utilizează recuperare semantică densă. Weaviate necesită cel puțin 16 GB RAM recomandat pentru autogăzduire și impune o schemă predefinită a colecției înainte de ingestia documentelor, adăugând complexitate nejustificată pentru cerințele proiectului.

3.2.7 Scanare web

Pentru scanarea web s-a utilizat Scrapy, împreună cu integrarea scrapy-redis pentru deduplicarea URL-urilor prin Redis. Crawllee este orientat în primul rând spre site-uri cu conținut generat dinamic prin JavaScript, oferind integrare nativă cu Playwright; deoarece site-urile facultății furnizează conținut HTML static, această capabilitate nu aduce niciun avantaj. Ecosistemul de middleware al Scrapy permite procesarea în etape a datelor colectate (validare, deduplicare, fragmentare, stocare), corespunzând exact arhitecturii secvențiale a proiectului. Varianta completă de scrapy-redis nu a fost adoptată din cauza lipsei de mentenanță activă; s-a utilizat exclusiv componenta de deduplicare prin Redis. Sistemul respectă automat fișierele robots.txt și permite personalizarea domeniilor urmărite.

3.2.8 Extragerea conținutului HTML

Pentru extragerea conținutului din paginile HTML s-a utilizat Trafilatura, o bibliotecă specializată în identificarea conținutului principal al unei pagini web și eliminarea elementelor de navigare, anteturi și conținut duplicat. Rezultatul poate fi returnat în mai multe formate: Markdown, text simplu sau XML. Trafilatura a fost preferată față de BeautifulSoup pentru performanța ridicată de extragere a textului relevant de pe site-urile universității, în condițiile în care structura HTML a surselor este neuniformă și nu permite aplicarea unor reguli de selecție fixe.

3.2.9 Procesarea documentelor structurate

Pentru procesarea documentelor structurate s-a utilizat Docling prin interfața DoclingLoader din LangChain. Fișierele DOCX sunt procesate direct, iar fișierele DOC sunt convertite în prealabil la DOCX prin LibreOffice; conversia nu influențează textul din document, ci doar

formatul de container. Fișierele XLSX sunt procesate cu `openpyxl`, iar conținutul tabular este convertit în Markdown printr-un apel la Claude API înainte de fragmentare, păstrând structura relațională a datelor din tabel.

3.2.10 Infrastructura cloud

Infrastructura cloud este construită pe Microsoft Azure. Alegerea față de AWS sau Google Cloud a fost motivată de programul Azure for Students, care oferă \$100 credit anual pe baza adresei de e-mail instituționale, eliminând costul operării pe durata dezvoltării. Toate serviciile utilizate sunt native Azure și se administrează într-un singur grup de resurse cu un model de autentificare comun bazat pe Managed Identity. Serviciile componente, rolul fiecăruia și punctul de integrare în proiect sunt descrise în detaliu în Secțiunea 3.3.

3.2.11 Deduplicarea URL-urilor

Redis este utilizat, prin integrarea `scrapy-redis`, ca filtru de deduplicare a URL-urilor în cursul procesului de scanare web. La fiecare pornire a agentului de colectare, cheile de deduplicare sunt golite explicit pentru a garanta o scanare completă a tuturor surselor. Față de o structură de date în memorie Python, Redis permite separarea stării de deduplicare de procesul principal, pregătind arhitectura pentru o parcurgere distribuită pe mai multe instanțe.

Tabelul 1 sintetizează alegerile tehnologice principale și alternativele considerate.

3.3 Infrastructura cloud Azure

Infrastructura cloud a sistemului cuprinde unsprezece servicii Microsoft Azure, organizate după responsabilitatea fiecăruia în cadrul arhitecturii.

Azure Table Storage asigură persistența datelor utilizatorilor: conturile (cu parola stocată sub formă de hash), istoricul conversațiilor și sursele accesate de toți utilizatorii platformei, înregistrate fiecare într-un tabel specific, partiționate după numele de utilizator.

Azure Key Vault centralizează toate secretele aplicației, inclusiv cheia API care apelează Claude, URL-ul Azure Redis, conexiunea către stocarea în Azure Blob Storage, secretul HuggingFace și șirul de caractere care definește conexiunea pentru serviciul de e-mail; toate sunt citite la pornire prin biblioteca `azure-identity`, fără a fi prezente în codul sursă sau în fișierele de configurare.

Azure Cache for Redis care furnizează coada de URL-uri și filtrul de deduplicare pentru `scrapy-redis`. La fiecare pornire a scanării web a surselor oficiale, cheile de deduplicare sunt golite explicit pentru a garanta o scanare completă a tuturor surselor de fiecare dată.

Table 1: Comparație tehnologii alese față de alternative

Componentă	Tehnologie aleasă	Alternative considerate	Motiv principal
Limbaj de programare	Python	TypeScript	Ecosistem IA exclusiv Python
Bază de date vectorială	ChromaDB	Pinecone, Elasticsearch, Weaviate	Cost și complexitate reduce
Infrastructură cloud	Microsoft Azure	AWS, Google Cloud	Credit Azure for Students
API model lingvistic	Claude API	OpenAI, Google Gemini	Fereastră de context extinsă
Extragere conținut HTML	Trafilatura	BeautifulSoup	Performanță pe site-uri nestructurate
Interfață utilizator	Streamlit	React Native	Integrare nativă cu Python
Manager pachete	uv	pip	Viteză și fișier lock reproductibil
Scanare web	Scrapy	Crawlee	Site-uri statice

Azure Blob Storage stochează fișierele descărcate de agentul de colectare sub forma unui format JSON. Un JSON vechi, de la vechea parcurgere și un fișier JSON nou. În fiecare JSON se va stoca temporar cheia hash a documentelor parcurse pentru a vedea schimbări de text.

Azure Container Apps găzduiește aplicația web Streamlit. Deoarece modelul CrossEncoder de re-ierarhizare este încărcat în memorie la pornire, fiecare instanță necesită cel puțin 2 vCPU și 4 GiB de memorie.

Azure Container Apps Jobs rulează agentul de colectare Scrapy conform unui program cron configurabil; procesul pornește, parcurge sursele configurate, scrie rezultatele în ChromaDB și se oprește. Ambele componente partajează același *Container Apps Environment*.

Azure Files este un spațiu de fișiere din cadrul aceluiași cont de stocare, montat ca volum SMB. Directorul bazei de date vectoriale ChromaDB și fișierele modelelor HuggingFace pre-antrenate sunt stocate pe acest volum și rămân persistente între repornirile instanțelor.

Azure Container Registry stochează imaginile Docker ale aplicației web și ale agentului de colectare. Accesul la registru este asigurat prin intermediul identității gestionate specifice Azure, fără credențiale stocate în configurație.

Azure Managed Identity este o identitate de sistem asignată mediului Container Apps și acordă accesul la Key Vault și registrul de containere prin controlul bazat pe roluri de access Azure (RBAC), utilizând clasa `DefaultAzureCredential` din `azure-identity`.

Azure Communication Services, împreună cu resursa Azure Email Communication Service

asociată, asigură livrarea e-mailurilor pentru fluxurile de recuperare a parolei și de confirmare a înregistrării la prima accesare a platformei. Resursa de e-mail poate fi configurată cu un subdomeniu gestionat de Azure (*.azurecomm.net), fără a necesita un domeniu propriu.

Azure VPN Gateway asigură conectivitatea securizată între rețeaua virtuală Azure și rețelele externe. Serviciul de VPN este necesar pentru componenta Azure Redis for Cache, nedorind expunerea adresei bazei de date Redis în mod public.

Tabelul 2 sintetizează serviciile utilizate și punctul de integrare al fiecăruia în proiect.

Table 2: Serviciile Azure utilizate și integrarea cu proiectul

Serviciu Azure	Rol în sistem	Punct de integrare
Azure Table Storage	Persistența conturilor de utilizator și a conversațiilor	src/azure/db/
Azure Key Vault	Gestionarea centralizată a secretelor	src/azure/kv/
Azure Cache for Redis	Coada URL-urilor și filtrul de deduplicare pentru agentul de colectare	src/spider/settings.py
Azure Blob Storage	Stocarea fișierelor descărcate de agentul de colectare înaintea procesării	FILES_STORE (pipeline Scrapy)
Azure Container Apps	Găzduirea aplicației web Streamlit	src/UI/entry_point.py
Azure Container Apps Jobs	Rularea periodică a spider-ului ca proces cron	src/spider/
Azure Files	Persistența ChromaDB și a modelelor Hugging-Face	src/vector_database/constants.py
Azure Container Registry	Stocarea imaginilor Docker ale aplicației și agentului de colectare	Imagine Docker a aplicației
Azure Managed Identity	Autentificare fără credențiale între serviciile Azure	DefaultAzureCredential
Azure Communication Services	Livrarea e-mailurilor tranzacționale	src/azure/ (modul dedicat)
Azure VPN Gateway	Conectivitate securizată între rețeaua virtuală și rețele externe (VNet)	

3.4 Integrare cu UPB Login API

Lucrarea de față fiind proiectată pentru posibilitatea unei utilizări ulterioare în producție, s-a ales autentificarea prin UPB Login API la fiecare accesare a utilizatorului în cazul unei sesiuni noi, definită ca trecerea a cel puțin unei ore de la ultima accesare. Această decizie asigură că

utilizatorii platformei sunt exclusiv membri ai comunității universitare UPB.

Integrarea a fost posibilă datorită punctelor de acces expuse de instanța Keycloak a UPB, un server de autentificare și autorizare care implementează protocolul OAuth 2.0. Punctul de acces ales este utilizat strict pentru autentificarea și validarea utilizatorului în baza de date UPB, fără a accesa sau stoca date suplimentare despre contul instituțional.

Odată utilizatorul validat de serverul Keycloak, platforma creează un cont și asigură atomicitatea utilizatorului: fiecare persoană va deține un singur cont pe platformă, identificat prin adresa instituțională, lucru garantat structural prin mecanismul de înregistrare prin Login UPB.

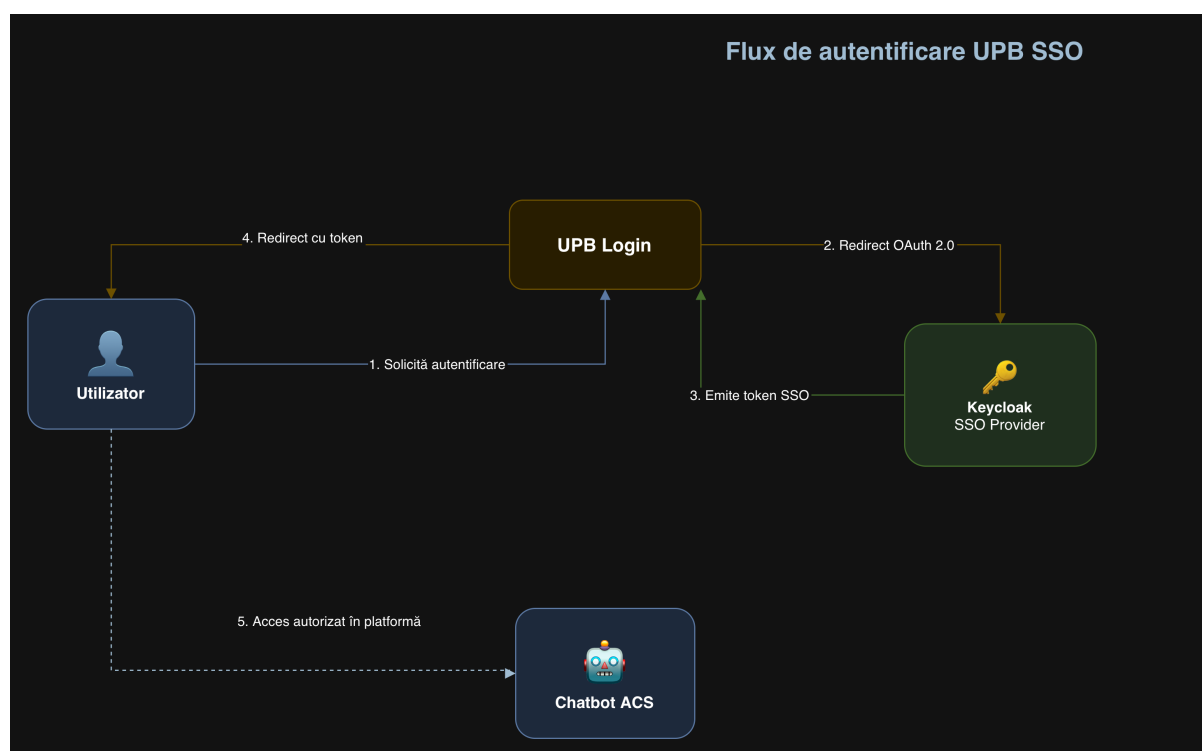


Figure 3: Fluxul de autentificare prin UPB Login API (OAuth 2.0 / Keycloak)

3.5 Fluxul de date

Interogarea brută este normalizată Unicode și rescrisă printr-un apel la Claude API care corectează diacriticele lipsă și erorile ortografice, asigurând că interogarea este specifică pentru un domeniu academic. Dacă interogarea are structură compusă, un al doilea apel la același model o descompune într-o listă de sub-interogări, fiecare cu semnificație semantică de sine stătătoare.

De exemplu, interogarea „*Cine e proful de la Baze de Date și câte credite are materia?*” este descompusă în: „*Care este profesorul titular al cursului Baze de Date?*” și „*Câte credite are materia Baze de Date?*”. Această descompunere permite recuperarea independentă a

fragmentelor relevante pentru fiecare aspect al întrebării originale.

Pentru fiecare sub-interogare, Claude API generează un document ipotetic în stilul unui document oficial al facultății folosind HyDE (*Hypothetical Document Embedding*). Vectorul corespunzător acestui document ipotetic este utilizat pentru interogarea bazei de date vectoriale ChromaDB. Fragmentele recuperate sunt filtrate după un prag de confidență, eliminând rezultatele cu relevanță scăzută, iar candidații rămași sunt reclasificați prin CrossEncoder după scorul de relevanță calculat prin atenție mutuală între interogare și fiecare fragment.

Primele N fragmente selectate, împreună cu interogarea originală, istoricul conversației și profilul utilizatorului, sunt transmise Claude API printr-o instrucțiune de sistem configurată specific pentru domeniul facultății. Răspunsul este redat în interfață către utilizator însoțit de referințele surselor din care au fost extrase fragmentele utilizate.

4 IMPLEMENTARE

4.1 Setările inițiale pentru contul de Azure Student

Accesul la infrastructura Azure se obține prin programul Azure for Students, disponibil pe baza adresei de e-mail instituționale (@stud.acs.upb.ro).

(Resource Group), reprezintă o componentă integratoare care organizează serviciile Azure asociate aceleiași aplicații. Grupul de resurse este creat o singură dată și se definește prin intermediul regiunii. Toate serviciile create ulterior în cadrul acelui grup moștenesc regiunea și pot fi administrate, monitorizate sau șterse împreună. Pentru proiectul de față s-a utilizat regiunea Poland Central.

Identitatea gestionată (Managed Identity) este activată la nivelul mediului Container Apps și acordă acces la celelalte resurse din același grup prin intermediul controlului bazat pe roluri, fără a stoca credențiale în codul sursă sau în fișierele de configurare. Această configurare se realizează o singură dată, prin atribuirea rolurilor corespunzătoare fiecărui serviciu.

4.2 Colectarea datelor prin scanare web

4.2.1 Componentele Scrapy folosite

Principalele elemente din Scrapy folosite în cadrul proiectului sunt: CrawlSpider, Rule, Response, HtmlResponse, ItemLoader, Item, LinkExtractor, CrawlerProcess, Settings, MapCompose

deosebite de clasa Spider, care necesită gestionarea manuală a cererilor de urmărire, CrawlSpider aplică automat regulile definite asupra fiecărui răspuns primit și generează cereri noi pentru legăturile care satisfac criteriile.

Rule definește o regulă de urmărire a legăturilor: asociază un LinkExtractor cu un apel recursiv de funcție care procesează paginile descoperite. Fiecare regulă poate fi configurată să indice dacă pagina găsită trebuie transmisă apelului recursiv sau doar trebuie urmărită pentru extragerea de noi legături.

LinkExtractor extrage legăturile din răspunsul HTML pe baza unor criterii configurabile: domenii permise, expresii regulate aplicate pe URL, selectori CSS. În cadrul proiectului, LinkExtractor este parametrizat cu lista domeniilor facultății, astfel că agentul va urmări numai legături interne, specifice domeniului țintă.

`Response` este obiectul de bază returnat de Scrapy la descărcarea unei resurse. Conține URL-ul final al cererii, codul de stare HTTP și textul răspunsului. Toate apelurile recursive din agent primesc un obiect de tip `Response` sau o subclasă a acestuia.

`HtmlResponse` este subclasa `Response` specializată pentru conținut HTML. Adaugă metodele `.css()` și `.xpath()` care permit interogarea directă a arborelui DOM prin selectori, fără a instanția manual un generator de conținut HTML.

`Item` este o structură de date specifică Scrapy. Structurează câmpurile extrase (URL, titlu, conținut, metadata) într-un obiect cu câmpuri declarate explicit, similar unui dicționar cu chei predefinite. Utilizarea `Item` față de un dicționar simplu permite validarea câmpurilor și compatibilitatea cu fluxul de prelucrare Scrapy.

`ItemLoader` populează un obiect de tip `Item` cu datele extrase din fiecare câmp, astfel că transformările (curățare text, normalizare URL, eliminare spații) sunt aplicate automat la atribuirea valorilor, fără a necesita cod explicit de curățare în apelul recursiv.

`MapCompose` reprezintă un preprocesor de date folosit în `ItemLoader` care aplică secvențial o listă de funcții asupra fiecărei valori unui câmp. De exemplu, `MapCompose(str.strip, str.lower)` aplică mai întâi eliminarea spațiilor, apoi conversia la litere mici, pentru fiecare valoare primită de câmpul respectiv.

`Settings` este obiectul de configurare al crawlerului. Definește parametri globali precum `ROBOTSTXT_OBEY`, `DOWNLOAD_DELAY`, middleware-urile active și fluxurile de procesare. Setările pot fi suprascrise per instanță de agent, permițând comportamente diferite pentru colectori diferiți din același proiect.

`CrawlerProcess` permite rularea agentului de colectare dintr-un script Python obișnuit, fără a apela comanda `scrapy crawl` din terminal. Permite transmiterea unui obiect `Settings` la pornire, ceea ce face posibilă integrarea crawlerului în fluxul de date al aplicației principale.

4.2.2 Setările Scrapy aplicate

Configurarea agentului de colectare este definită în fișierul `settings.py` și controlează comportamentul de rețea, nivelul de paralelism și strategia de reîncercare a URL-urilor.

`ROBOTSTXT_OBEY = True` îi spune agentului să citească și să respecte fișierul `robots.txt` al fiecărui domeniu vizitat înainte de a trimite cereri. Paginile excluse prin acest fișier nu sunt accesate, indiferent dacă apar ca legături în paginile care urmează să fie parcurse.

Paralelismul este configurat prin `CONCURRENT_REQUESTS = 64` și `CONCURRENT_REQUESTS_PER_DOMAIN = 32`, ceea ce permite descărcarea simultană a mai multor pagini.

Ritmul `AutoThrottle` ajustează dinamic intervalul dintre cereri în funcție de timpul de răspuns al serverului. Configurația utilizată definește o întârziere inițială de 0,25 secunde (`AUTOTHROTTLE_START_DELAY`), o întârziere maximă de 3 secunde (`AUTOTHROTTLE_MAX_DELAY`)

și o concurență țintă de 8 cereri simultane (`AUTOTHROTTLE_TARGET_CONCURRENCY`). Agen-
tul reduce viteza de colectare atunci când serverul răspunde lent și o crește atunci când
răspunsurile sunt rapide.

Reîncercarea automată este activată (`RETRY_ENABLED = True`) cu maximum două încercări
pentru codurile de eroare HTTP asociate: 500, 502, 503, 504, 408, 421 și 429.

4.2.3 Generarea unui agent utilizator dinamic

Site-urile web pot bloca cererile care nu prezintă antetele specifice unui browser real. Un agent
de colectare care trimite șirul `User-Agent` implicit al bibliotecii HTTP poate fi identificat ca
robot și poate fi blocat sau redirecționat. Pentru a evita această situație, fiecare cerere
primește un set complet de antete generate dinamic, care reproduce profilul unui browser de
desktop.

Versiunile minime acceptate pentru fiecare browser (Chrome, Firefox, Edge, Browser. Bib-
lioteca `ua_generator` primește aceste versiuni minime prin obiectul `Options` ceea ce face ca
versiunile mai recente să fie selectate cu probabilitate mai mare. La fiecare apel, biblioteca
produce un șir `User-Agent` valid, corespunzător unui browser de desktop ales aleatoriu din
mulțimea celor patru browsere configurate.

Pe lângă șirul `User-Agent`, fiecare cerere primește un antet `accept-language` ales aleatoriu,
precum și un set de antete statice care reproduc comportamentul unui browser real: `accept`,
`accept-encoding`, `upgrade-insecure-requests` și antetele `sec-fetch-*`.

Injectarea antetelor este realizată de `RotatingHeadersMiddleware`, un `downloader middle-
ware` Scrapy. Metoda `process_request` este apelată pentru fiecare cerere înainte de trimitere
și suprascrie antetele cu un set nou generat, astfel că cererile consecutive către același server
nu o să prezinte același `User-Agent`.

4.2.4 Folosirea unor certificate SSL vechi

Unele site-uri ale facultății utilizează certificate SSL vechi sau auto-semnate, respinse de
clienții TLS moderni cu nivelul de securitate implicit. Clasa `CustomContextFactory` extinde
`ScrapyClientContextFactory` și dezactivează verificarea certificatului (`verify=False`),
reducând totodată nivelul de securitate. Reducerea securității este necesară pentru a accesa
sursele facultății.

4.2.5 Agentul de colectare web Scrapy

`PageSpider` extinde `CrawlSpider` și primește la inițializare lista URL-urilor de start din care
derivă automat domeniile permise (`allowed_domains`). Se vor accepta numai URL-uri din

același domeniu exact și se vor exclude paginile cu parametri de interogare, paginile în limba engleză (/en/), apelurile AJAX și URL-urile care conțin ani anteriori anului curent minus doi, filtrate printr-o expresie.

Apelul recursiv `parse_page` extrage titlul paginii, conținutul Markdown, structura de titluri HTML (h1, h2, h3) și tabelele, construiește un `PageItem` și declanșează extragerea documentelor atașate prin `_extract_documents`.

4.2.6 Extragerea documentelor

Metoda `_extract_documents` parcurge toate elementele `<a>` și `<area>` din răspunsul HTML și selectează URL-urile cu extensiile `.pdf`, `.xls`, `.xlsx`, `.doc` și `.docx`, cu condiția că aparțin domeniului permis. Pentru fiecare URL selectat se produce un `DocumentItem` care este preluat de `DocumentFilesPipeline`.

`DocumentFilesPipeline` descarcă fișierul cu un timp de așteptare de 200 de secunde, apelează `parse_document` cu calea locală și URL-ul sursei, curăță fragmentele rezultate și le stochează în ChromaDB cu cheia SHA-256 ai conținutului.

4.2.7 Extragerea textului

Conținutul text al fiecărei pagini este extras prin `Trafilatura` în două formate: text simplu, utilizat pentru calculul hash-ului SHA-256 al paginii, și Markdown, utilizat pentru fragmentare și indexare. Deoarece `Trafilatura` elimină etichetele `<h1>`, `<h3>` din conținutul extras, agentul le va stoca separat prin selectori CSS și le folosește în conținutu; Markdown, reconstituind structura de titluri înainte de fragmentare. Structura de titluri este necesară pentru o contextualizare mai bună în ChromaDB a fragmentelor.

4.2.8 Extragerea tabelelor

Tabelele HTML sunt extrase separat de conținutul text prin `select_tables_from_plain_html`, care selectează elementele `<table>` din HTML-ul brut al răspunsului. `tables_to_markdown` convertește fiecare tabel în format Markdown. Fragmentele sunt stocate în ChromaDB independent de fragmentele de text, separate printr-un delimitator, astfel că segmentarea nu poate apărea la granița dintre text narativ și conținut tabelar.

4.2.9 Fluxul de prelucrare a datelor

Datele colectate trec prin patru etape aplicate secvențial, în ordinea priorităților configurate.

ValidationPipeline (prioritate 100) verifică prezența URL-ului și îl normalizează. DeduplicationPipeline (200) elimină elementele cu URL deja văzut sau cu hash de conținut duplicat, menținând două seturi în memorie pe durata rulării. DocumentFilesPipeline (300) descarcă, transformă și indexează documentele. ContentChunkingPipeline (400) segmentează conținutul Markdown în fragmente de text și fragmente de tip tabel, aplică deduplicare exactă prin SHA-256 și deduplicare aproximativă prin MinHash LSH și stochează în ChromaDB numai fragmentele unice.

4.3 Procesarea documentelor și a textului

4.3.1 Transformarea documentelor cu Docling și strategia de segmentare

Documentele descărcate (PDF, DOCX, DOC, XLSX, XLS) sunt procesate prin DocumentConverter din biblioteca Docling. Pentru documentele de tip XLSX și XLS s-a folosit un apel către Claude API. Suportul pentru segmentarea acestor tipuri de documente este limitat.

Segmentarea se realizează cu HybridChunker din docling-core, instanțiat cu același segmentator ca modelul de codificare (multilingual-e5-large), cu limita de 512 cuvinte pe fragment. Alinierea segmentatorului cu modelul de codificare garantează că niciun fragment nu depășește fereastra de context a modelului la codificare. Metoda contextualize aplicată asupra unei bucăți de document adaugă contextul structural relevant (titlu de secțiune, poziție în ierarhie) la conținutul fiecărui fragment.

4.3.2 Transformarea textului și strategia de segmentare

Conținutul HTML extras de Trafilatura în format Markdown este procesat prin același DocumentConverter. Alegerea aceluiași convertor pentru paginile web și pentru documente asigură o strategie de segmentare uniformă pentru tot corpusul.

Fiecare fragment primit trece prin funcția `enrich_chunk`, care prepune trei prefixe la conținutul textual: `[Titlu: ...]`, `[Secțiune: ... > ...]`, construită din ierarhia de titluri a paginii, și `[Cuvinte cheie: ...]`, extrase din subdomeniul și cuvintele care sunt în calea URL-ului sursei. Aceste prefixe sunt stocate direct în câmpul `page_content` al fragmentului, nu în metadata, astfel că modelul de codificare le include în reprezentarea vectorială și îmbunătățesc scorul de similitudine la interogare.

De exemplu, un fragment extras din pagina evenimentelor departamentului AII este stocat în ChromaDB cu structura:

```
[Cuvinte cheie: aii, mec-events,
```

```
scoala-de-vara-automatizare-procese-software-2026]
[Section: Scoala de vara - Automatizare Procese Software
    > Informatii generale > Instructori]
[Titlu: scoala de vara - automatizare procese software
    - departamentul aii]
```

```
Scoala de vara - Automatizare Procese Software
Informatii generale > Instructori
- Narcis-Alfred SZENE (UiPath, doctorand UPB)
- Victor-Valentin ANGHRL (doctorand UPB)
```

Fără aceste prefixe, conținutul „*Narcis-Alfred SZENE (UiPath, doctorand UPB)*” ar produce un vector fără context, irelevant pentru o interogare de tipul „*Cine predă la școala de vară de automatizare?*”.

4.4 Codificare și indexare

Fragmentele de text și de tabele sunt codificate cu modelul `intfloat/multilingual-e5-large`. Fiecare fragment este stocat cu un identificator SHA-256 al conținutului; un identificator deja prezent nu va fi inserat, ceea ce face operația idempotentă și elimină nevoia de verificare ulterioară.

La prima interogare prin cuvinte cheie se construiește în memorie un index BM250kapi peste toate documentele din colecție, citite integral din ChromaDB. BM25 ponderează automat termenii prin IDF, compensând limitele recuperării dense în cazul termenilor tehnici sau al numelui propriu cu frecvență scăzută în corpus.

4.5 Lanțul de recuperare (RAG)

4.5.1 Recuperarea densă

Recuperarea densă se realizează prin `similarity_search_with_relevance_scores` al interfeței Chroma din LangChain, care returnează perechile (document, scor cosinus) pentru cei mai apropiați k vectori din index. Funcția acceptă un filtru opțional pe metadatele `url_slug`, utilizat pentru prioritizarea surselor accesate anterior în aceeași conversație. Sunt recuperați cel mult 40 de candidați, din care sunt reținuți cei cu scor de cel puțin 0,60.

4.5.2 Re-ierarhizarea cu cross-encoder

Candidații rămași sunt reclasificați de modelul BAAI/bge-reranker-v2-m3, un cross-encoder care procesează simultan perechea (interogare, fragment) prin mecanismul de atenție mutuală. Spre deosebire de modelul de codificare, care codifică interogarea și documentele independent, cross-encoderul calculează relevanța prin compararea directă a celor două texte, obținând scoruri mai precise. Primele 10 fragmente după reclasificare sunt transmise generatorului.

4.5.3 Reformularea interogărilor

Interogarea brută trece prin trei transformări aplicate secvențial. Prima este normalizarea Unicode (forma NFC) cu conversia la litere mici, necesară pentru coerența semantică. Dacă interogarea curentă conține cuvinte care sunt similare cu termeni din istoricul conversației, se va face un apel la Claude API prin care cuvintele vor fi înlocuite cu contextul semantic complet. A treia este rescrierea academică: un al doilea apel corectează diacriticele lipsă, erorile ortografice și abrevierile. Rezultatele sunt stocate printr-o memorie tampon cu 256 de intrări pentru a evita apeluri redundante la API.

4.5.4 Atomicizarea interogărilor

Dacă interogarea conține conjuncții coordonatoare care unesc două cereri de informații distincte, un apel la `claude-sonnet-4-6` returnează o listă JSON de sub-interogări, fiecare completă și de sine stătătoare. `claude-sonnet-4-6` este ales în locul modelului Haiku pentru această sarcină deoarece decompoziția greșită a unei interogări propagă erori prin tot lanțul de recuperare. Rezultatele sunt memorate printr-un cache LRU cu 256 de intrări. Fiecare sub-interogare este procesată independent prin restul lanțului de recuperare.

De exemplu, interogarea „*Cine e proful de la Baze de Date și câte credite are materia?*” este descompusă în:

```
[
  "Care este profesorul titular al cursului Baze de Date?",
  "Câte credite are materia Baze de Date?"
]
```

Fără descompunere, vectorul interogării compuse ar fi un compromis între cele două aspecte și ar recupera documente irelevante pentru ambele.

4.5.5 Generarea unor documente sintetice

Pentru fiecare sub-interogare, în loc să se utilizeze vectorul interogării rescrise, Claude API generează un paragraf de 3, 5 propoziții în stilul paginilor oficiale ale facultății, conform metodei HyDE (*Hypothetical Document Embedding*). Vectorul acestui document ipotetic se află în același subspațiu semantic cu documentele indexate.

De exemplu, pentru interogarea „*Câte credite are materia Sisteme de Operare*”, documentul ipotetic generat este:

Disciplina Sisteme de Operare, inclusă în planul de învățământ al programelor de studiu din cadrul Facultății de Automatică și Calculatoare, are o valoare de 6 credite ECTS. Această disciplină obligatorie se desfășoară pe parcursul unui semestru și cuprinde activități de prelegere, seminar și laborator, conform fișei disciplinei aprobate de catedră. Creditul alocat reflectă volumul de muncă estimat pentru student, incluzând atât activitățile didactice directe, cât și studiul individual necesar pentru asimilarea competențelor specifice.

Vectorul acestui paragraf este semantic mai apropiat de fișele de disciplină indexate în ChromaDB decât vectorul interogării colocviale originale, deoarece reproduce terminologia și stilul documentelor sursă.

4.6 Detectarea schimbării documentelor

La fiecare rulare, agentul de colectare stochează în Azure Blob Storage un fișier JSON cu hash-urile SHA-256 ale documentelor procesate. La rularea următoare, hash-urile curente sunt comparate cu cele din fișierul anterior; documentele cu hash modificat sunt reindexate, iar cele fără modificări nu sunt procesate din nou. Mecanismul reduce volumul de date scrise în ChromaDB la fiecare rulare zilnică și limitează numărul de apeluri la API pentru documentele nemodificate.

4.7 Generarea răspunsurilor

Generarea răspunsului este realizată de `get_chatbot_response` prin `claude-sonnet-4-6`. Fragmentele recuperate sunt formate ca `[Sursă N]: conținut` și concatenate cu interogarea utilizatorului într-un singur mesaj de tip `user`. Instrucțiunea de sistem specifică domeniul facultății și interzice explicit generarea de informații din afara fragmentelor furnizate.

Profilul studentului (`user_context`: an de studiu, departament, grupă) și rezumatul conversației curente sunt injectate în instrucțiunea de sistem. Din istoricul conversației sunt incluse ultimele șase replici, suficiente pentru coerența conversațională.

4.8 Gestionarea contextului

Contextul utilizatorului este construit la fiecare interogare din datele stocate în sesiune la autentificare: an de studiu, departament și grupă, mapate la denumirile oficiale ale specializărilor (Ingineria Sistemelor, Calculatoare și Tehnologia Informației). Contextul este transmis atât funcției de recuperare, unde filtrează sursele relevante pentru specializarea studentului, cât și metodei de răspuns, unde ajustează formularea răspunsului.

Rezumatul conversației curente este citit din Azure Table Storage și injectat în instrucțiunea de sistem la fiecare interogare. Rezumatul înlocuiește transmiterea întregului istoric în sistemul de generare, reducând numărul de credite consumate fără a pierde continuitatea conversației.

4.9 Gestionarea conversațiilor

Fiecare conversație este stocată în Azure Table Storage cu un identificator unic, titlu, rezumat, lista de mesaje și momentul la care a fost creat. La încheierea unei conversații, `summarize_conversation` apelează Claude API cu `tool_use(tool_choice: {type: tool})` pentru a obține structurat un titlu de maximum 5 cuvinte și un rezumat de 2, 3 propoziții. Utilizarea `tool_use` în loc de generare liberă garantează că răspunsul respectă structura așteptată fără adăugări suplimentare.

De exemplu, la finalul unei conversații despre credite și examene, `summarize_conversation` produce prin `tool_use`:

```
{
  "title": "Credite și examene restanță",
  "summary": "Studentul a întrebat despre numărul de credite al
             materiei Baze de Date și despre procedura de depunere
             a cererii de reexaminare. Sistemul a furnizat
             informațiile din sursele oficiale ale facultății."
}
```

Structura JSON este garantată de `tool_choice: {type: tool}`, care forțează modelul să apeleze unealta `summarize_conversation` în loc să genereze text liber, eliminând nevoia de transformare a răspunsului.

Conversațiile au o durată de viață configurabilă; după expirare, câmpul de introducere este dezactivat și studentul este îndemnat să înceapă o conversație nouă. Lista conversațiilor anterioare este afișată în bara laterală și se actualizează după fiecare conversație salvată.

4.10 Sistemul de autentificare

Autentificarea se bazează pe un sistem local cu nume de utilizator și parolă stocată ca hash în Azure Table Storage. La autentificare reușită, sesiunea este persistată printr-un cookie de browser, evitând reautentificarea ulterioară

Fluxul de recuperare a parolei trimite un email prin Azure Communication Services cu un cod de resetare temporar. Conturile sunt create prin autentificarea cu UPB Login API, care garantează că accesul este restricționat membrilor comunității universitare.

4.11 Interfața utilizator

Interfața este construită cu Streamlit și cuprinde pagina de conversație, bara laterală cu istoricul conversațiilor și paginile de autentificare. Răspunsul modelului este afișat cuvânt cu cuvânt prin `st.write_stream`, fără a bloca interfața până la finalizarea completă. Sub fiecare răspuns, un panou care se extinde, afișează URL-urile surselor din care au fost extrase fragmentele utilizate, permițând studentului să verifice sursa primară.

Starea interfeței este gestionată prin `st.session_state`. Un indicator `is_waiting` blochează câmpul de introducere pe durata procesării pentru a preveni trimiteri simultane. Interogarea curentă este stocată în `pending_prompt` și procesată la rerularea scriptului declanșată de Streamlit, separând primirea inputului de procesarea acestuia.

5 PROVOCĂRI ÎNTÂMPINATE

5.1 Provocări legate de fluxul de recuperare al datelor

În acest moment, toate datele sunt stocate în baza de date vectorială ChromaDB, iar interogarea se face asupra tuturor datelor prezente, selectând cea mai bună variantă. Inițial, aplicația utiliza o altă metodă de abordare, anume maparea URL-urilor în anumite categorii, fiecare având un set de cuvinte cheie; fiecare URL asocia cuvintele cheie corespunzătoare bazate pe conținut. Sursele se identificau în momentul interogării pe baza similarității dintre întrebarea utilizatorului și conținutul prezent în surse.

Spre exemplu, sursa: <https://aii.pub.ro/> conținea categoria Ingineria Sistemelor, care includea cuvinte cheie precum: Teoria Sistemelor Automate, ACS-IS, AII, Automatică Industrială, dar conținea și categorii comune tuturor surselor, precum: Administrativ, General, Evenimente.

Având în vedere lipsa structurii, repetarea multor cuvinte cheie și prezența mai multor categorii de-a lungul mai multor surse, șansele ca sursa dorită să fie relevantă erau reduse; prin urmare, erau alese sursele cu conținutul semantic cel mai apropiat.

Totodată, nu se găsea răspuns specific pentru interogări de tipul: Ce materie predă X? sau Cine este profesorul X? Metoda de abordare utilizată nu viza numele persoanelor sau interogările compuse.

Pentru a rezolva această problemă, s-a ales stocarea tuturor datelor în cadrul bazei de date vectoriale ChromaDB, care, la ingestie, asociază fiecărei bucăți de document cuvinte cheie precum: prefixul sursei, cuvintele din URL, titlul și subcapitolele.

Abordarea propusă rezolvă problema identificată; contextul fiecărei bucăți de text va fi îmbunătățit prin intermediul cuvintelor cheie adăugate.

5.2 Provocări în transformarea și normalizarea textului și a tabelelor

În ciuda unelei foarte atent alese, Trafilatura oferă rezultate bune pentru extragerea conținutului, dar nu este potrivită pentru integrarea tabelelor alături de restul textului prezent în pagină.

Problema principală a apărut atunci când textul era tratat ca un bloc unitar, combinând atât conținut narativ, cât și conținut tabelar: contextul semantic era pierdut, iar segmentarea se putea întâmpla între text și începutul tabelului.

Chiar dacă se ținea cont de cuvintele intermediare pentru a nu se pierde context, performanța a fost slabă în realizarea unor teste simple în care se îmbinau text și tabel.

Pentru această problemă s-a folosit o abordare hibridă: textul și conținutul de tip tabelar sunt procesate separat. Atât tabelul, cât și textul sunt extrase și segmentate independent, iar în urma segmentării, fragmentele rezultate sunt indexate diferit.

Totodată, având în vedere conținutul rigid prezent de-a lungul surselor folosite, s-a ales parametrizarea specifică uneltei Trafilatura cu parametrii: **favor precision = True** și **deduplicate = True** pentru a scăpa de zgomot.

5.3 Provocări în parcurgerea site-urilor web

Sursele conțin legături externe care pot fi indexate de către agentul de colectare, ieșind astfel din domeniul țintă propus.

Rezolvarea problemei a constatat în personalizarea unui reguli scrapy pentru a urma doar domeniile respective.

5.4 Optimizarea fluxului de recuperare

Baza de date vectorială ChromaDB indexează un volum mare de documente, ceea ce crește probabilitatea de recuperare a unor fragmente cu context semantic irelevant pentru interogarea curentă. Trei mecanisme suplimentare de filtrare au fost implementate pentru a reduce acest efect. Primul mecanism introduce un cache semantic la nivelul conversației. Fiecare pereche de replici (întrebare și răspuns) este codificată și stocată într-o bază de date vectorială separată. La o interogare nouă, similitudinea față de perechile anterioare este calculată; dacă scorul depășește un prag predefinit, răspunsul codificat este returnat direct, fără a interoga ChromaDB. Principiul este similar cache-ului semantic utilizat în arhitecturi de căutare de tip Elasticsearch și reduce costul computațional al interogărilor repetate în cadrul aceleiași sesiuni.

Al doilea mecanism utilizează istoricul resurselor accesate în conversație ca filtru de priorizare. URL-urile surselor furnizate ca referințe de răspuns sunt înregistrate sub forma unui dicționar cu structura (URL, frecvență de apariție). La interogările ulterioare, fragmentele provenite din URL-urile cu frecvența cea mai ridicată sunt căutate cu prioritate, reducând spațiul de căutare la sursele dovedite relevante pentru acel utilizator în cursul conversației.

Al treilea mecanism ordonează tipurile de surse după densitatea informațională. Facultatea de Automatică și Calculatoare pune la dispoziție documente în formate PDF, XLS, DOC, DOCX și XLSX. Deoarece informația relevantă se regăsește în cea mai mare parte în text continuu, căutarea este efectuată mai întâi în fragmentele de text indexate în ChromaDB; dacă acestea nu conțin informația necesară, sistemul trece la parcurgerea documentelor binare.

6 METODOLOGIE DE EVALUARE

Evaluarea sistemului se realizează pe un set de 65 de întrebări formulate manual, care acoperă opt categorii tematice. Fiecare întrebare vizează un tip de informație administrativă sau academică disponibilă în documentele oficiale ale facultății, colectate prin scanare web. Corectitudinea fiecărui răspuns generat este apreciată prin inspecție manuală. Răspunsul este citit și verificat în raport cu informațiile prezente în sursele indexate, fără utilizarea unui răspuns de referință predefinit. Un răspuns este considerat corect dacă conține informația relevantă din surse, indiferent de formularea lingvistică utilizată.

Cele opt categorii de întrebări sunt:

1. **Cadre didactice** (întrebările 1-10): titularii de curs sau de laborator pentru diverse discipline.
2. **Credite ECTS** (întrebările 11-20): numărul de credite alocat disciplinelor.
3. **Proceduri administrative** (întrebările 21-30): cereri, adeverințe, contestații și alte proceduri specifice secretariatului.
4. **Calendar academic** (întrebările 31-40): sesiuni de examene, termene și structura anului universitar.
5. **Plan de învățământ și specializări** (întrebările 41-50): structura programelor de studii și materiile obligatorii sau opționale.
6. **Întrebări combinate** (întrebările 51-55): întrebări care solicită simultan mai multe tipuri de informații, de exemplu titularul și creditele unei discipline.
7. **Urmărire conversațională** (întrebările 56-60): întrebări de tip follow-up formulate în contextul unei conversații anterioare, testate cu istoric de conversație injectat.
8. **Diverse** (întrebările 61-65): întrebări care nu se încadrează în categoriile anterioare.

6.1 Rezultatele evaluării

6.2 Limitări

Evaluarea manuală este parțial subiectivă în aprecierea răspunsurilor parțiale sau ambigue. Setul de 65 de întrebări nu acoperă toate tipurile de interogări posibile, iar rezultatele depind de calitatea și completitudinea corpusului indexat la momentul evaluării. Deoarece nu sunt utilizate răspunsuri de referință predefinite, verdictul de corectitudine se bazează pe inspecția directă a surselor indexate de către evaluator.

7 CONCLUZII ȘI MUNCĂ VIITOARE

7.1 Concluzii

Lucrarea de față a urmărit construirea unui agent conversațional pentru Facultatea de Automatică și Calculatoare, capabil să răspundă întrebărilor administrative ale studenților pe baza surselor oficiale ale facultății. Toate cele patru obiective stabilite în capitolul introductiv au fost atinse.

Primul obiectiv a constat în construirea unui sistem de recuperare ancorat în documentele oficiale ale facultății, cu răspunsuri verificabile. Sistemul utilizează o bază de date vectorială ChromaDB populată zilnic prin intermediul unui agent de colectare Scrapy, care colectează conținut din paginile web ale facultății și din documentele atașate în formate PDF, DOC, DOCX și XLSX. Fiecare răspuns generat este însoțit de referințele fragmentelor utilizate, permițând studentului să verifice sursa primară a informației.

Al doilea obiectiv a vizat reducerea discrepanței dintre limbajul colocvial al studenților și formalitatea documentelor oficiale. Modulul de preprocesare implementat corectează diacriticele lipsă și erorile ortografice, rescrie interogarea în registru academic și descompune cererile compuse în sub-interogări independente. Fiecare sub-interogare este augmentată prin generarea unui document ipotetic în stilul surselor facultății, conform metodei HyDE, înainte de interogarea bazei de date vectoriale.

Al treilea obiectiv a urmărit realizarea unei interfețe accesibile în afara orelor de program. Interfața web construită cu Streamlit este disponibilă permanent, gestionează istoricul conversațiilor cu titluri și rezumate generate automat și se integrează cu mecanismul de autentificare prin UPB Login API, garantând că platforma este accesibilă exclusiv membrilor comunității universitare.

Din perspectiva contribuțiilor tehnice, lucrarea propune un flux complet de recuperare pentru limba română în domeniul academic: normalizare lexicală, descompunere a interogărilor, generare de documente ipotetice prin HyDE și re-ierarhizare cu cross-encoder multilingv. Infrastructura de colectare a datelor, bazată pe Scrapy cu deduplicare prin Redis, permite actualizarea zilnică a corpusului fără intervenție manuală.

Evaluarea prezintă limitări de care trebuie ținut cont în interpretarea rezultatelor. Setul de 65 de întrebări nu acoperă toate tipurile de interogări posibile, iar aprecierea răspunsurilor parțiale este subiectivă. Corpusul indexat depinde de acoperirea agentului de colectare la momentul evaluării.

7.2 Muncă viitoare

7.2.1 Dezvoltarea unei interfețe utilizator moderne

Interfața actuală, construită cu Streamlit, răspunde cerințelor funcționale ale prototipului, dar prezintă limitări în personalizarea vizuală și în performanța la număr mare de utilizatori în același timp. O interfață construită folosind un limbaj specific dezvoltării web precum React sau Next.js ar permite implementarea unui design adaptat dispozitivelor mobile și separarea completă a interfeței față de logica aplicației.

7.2.2 Integrare cu toate site-urile web ale facultății

Agentul colector actual acoperă doar o parte din site-urile web oficiale ale Facultății de Automatică și Calculatoare. Extinderea corpusului la toate paginile afiliate ale facultății ar reduce riscul de răspunsuri incomplete cauzate de absența unor surse din index. Această extindere presupune identificarea și validarea tuturor domeniilor relevante, adaptarea regulilor de filtrare și evaluarea impactului volumului crescut asupra timpilor de interogare.

7.2.3 Infrastructură proprie față de API

Sistemul utilizează Claude API pentru toate apelurile de generare, reformulare și descompunere a interogărilor, ceea ce implică cost și dependența față de un serviciu extern. Rularea unui model propriu pe infrastructura facultății ar elimina aceste dependențe. Modele precum Llama 3 reprezintă alternative viabile pentru sarcinile de reformulare și generare de text scurt.

7.2.4 Integrare cu platforma Microsoft Teams

Microsoft Teams este platforma de comunicare utilizată cel mai des în cadrul Universității Politehnica București pentru coordonarea activităților didactice. De multe ori, documentele sau informațiile sunt puse pe canalul de comunicare de pe Teams. Implementarea unui robot Teams care să expună funcționalitățile sistemului ar permite studenților să adreseze întrebări administrative fără a accesa o platformă separată. Integrarea presupune înregistrarea aplicației în Azure Active Directory și presupune crearea unui cont fictiv, făcut exclusiv pentru a avea access la canalul de comunicare.

7.2.5 Integrare cu platforma de cursuri Moodle

Platforma Moodle este utilizată pentru distribuirea materialelor de curs, gestionarea temelor și comunicarea dintre studenți și cadre didactice. Accesul sistemului la conținutul Moodle prin

intermediul credențialelor de tip SSO utilizate de la Login UPB API ar extinde corpul cu resurse specifice fiecărui curs: anunturi, materiale de laborator, materiale de curs. Răspunsurile generate ar fi astfel mai precise pentru întrebările referitoare la conținutul academic, nu doar la procedurile administrative.

7.2.6 Integrare cu platforma studenti.pub.ro

Platforma studenti.pub.ro este utilizată pentru a centraliza informațiile academice personale ale studenților: situația școlară, crearea unui tichet la secretariat.. Integrarea cu această platformă ar permite sistemului să furnizeze răspunsuri personalizate, bazate pe situația academică a utilizatorului autentificat. Accesul ar trebui restricționat prin intermediul autentificării cu credențialele de la Login UPB astfel că sistemul să acceseze exclusiv datele studentului autentificat, fără a centraliza informații personale.

7.2.7 Detectare în timp real a schimbării conținutului documentelor

Mecanismul actual de actualizare se bazează pe un colector programat zilnic, care reindexează sursele monitorizate și schimbă baza de date vectorială. Această abordare introduce o latență de până la 24 de ore între publicarea unui document actualizat și reflectarea modificării în răspunsurile sistemului. Un mecanism bazat pe compararea hash-urilor de conținut la intervale mai scurte, sau pe notificări de tip webhook acolo unde sursa le suportă, ar garanta că informațiile furnizate corespund versiunii curente a documentelor oficiale.

7.2.8 Folosirea imaginilor în momentul generării răspunsului

Documentele facultății conțin frecvent informații prezentate sub formă de imagini. Versiunea actuală procesează exclusiv conținutul textual extras din documente, fără a include elementele vizuale. Integrarea capabilităților multimodale ale modelelor lingvistice moderne ar permite transmiterea imaginilor relevante ca parte a contextului de generare, astfel că informațiile prezentate grafic să fie accesibile în răspunsurile sistemului.

REFERINȚE

- K. Taneja, P. Maiti, S. Kakar, P. Guruprasad, S. Rao, and A. K. Goel, "Jill Watson: A Virtual Teaching Assistant powered by ChatGPT," in *Proc. 21st Int. Conf. Intelligent Tutoring Systems (ITS 2024)*, Springer, 2024. arXiv:2405.11070.
- G. Kortemeyer, "Ethel: A Virtual Teaching Assistant," *The Physics Teacher*, vol. 62, no. 8, AIP Publishing, 2024. arXiv:2407.19452.
- J. Weizenbaum, "ELIZA: A Computer Program for the Study of Natural Language Communication Between Man and Machine," *Commun. ACM*, vol. 9, no. 1, pp. 36–45, Jan. 1966.
- Y. Bengio, R. Ducharme, P. Vincent, and C. Jauvin, "A Neural Probabilistic Language Model," *J. Mach. Learn. Res.*, vol. 3, pp. 1137–1155, 2003.
- S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.
- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proc. 2019 Conf. North American Chapter Assoc. Comput. Linguist. (NAACL-HLT)*, pp. 4171–4186, Jun. 2019.
- A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models Are Unsupervised Multitask Learners," *OpenAI Blog*, vol. 1, no. 8, 2019.
- T. B. Brown, B. Mann, N. Ryder, et al., "Language Models Are Few-Shot Learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.
- J. Wei, Y. Tay, R. Bommasani, et al., "Emergent Abilities of Large Language Models," *Trans. Mach. Learn. Res.*, 2022.
- L. Ouyang, J. Wu, X. Jiang, et al., "Training Language Models to Follow Instructions with Human Feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.

-
- Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, 2020.
- V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, "Dense Passage Retrieval for Open-Domain Question Answering," in *Proc. 2020 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, pp. 6769–6781, ACL, 2020.
- L. Gao, X. Ma, J. Lin, and J. Callan, "Precise Zero-Shot Dense Retrieval without Relevance Labels," in *Proc. 61st Annu. Meeting Assoc. Comput. Linguist. (Volume 1: Long Papers)*, ACL, Toronto, Canada, Jul. 2023, pp. 1762–1777. doi:10.18653/v1/2023.acl-long.99.
- S. Bhat, A. G. Karegowda, V. S. Naidu, and C. S. Akshatha, "Leveraging HyDE and RAG in Gemma LLM Framework for Enlightened Deep Learning Chatbots," *Hum.-Cent. Intell. Syst.*, vol. 5, no. 4, pp. 467–481, 2025. doi:10.1007/s44230-025-00121-6.
- M. Najork and J. L. Wiener, "Breadth-First Crawling Yields High-Quality Pages," in *Proc. 10th Int. Conf. World Wide Web (WWW 2001)*, ACM, 2001, pp. 114–118.
- A. Z. Broder, "On the Resemblance and Containment of Documents," in *Proc. Compression and Complexity of Sequences (Sequences 1997)*, IEEE, 1997, pp. 21–29.